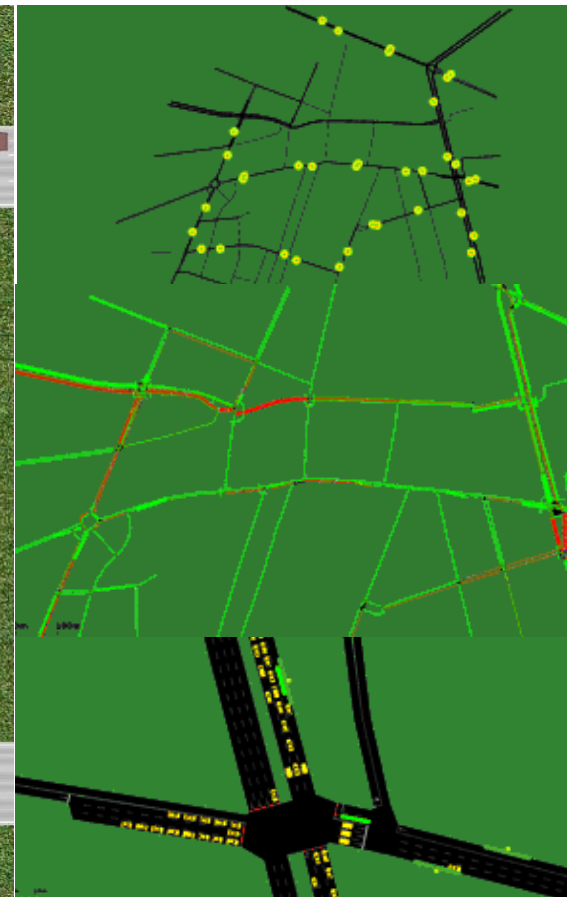
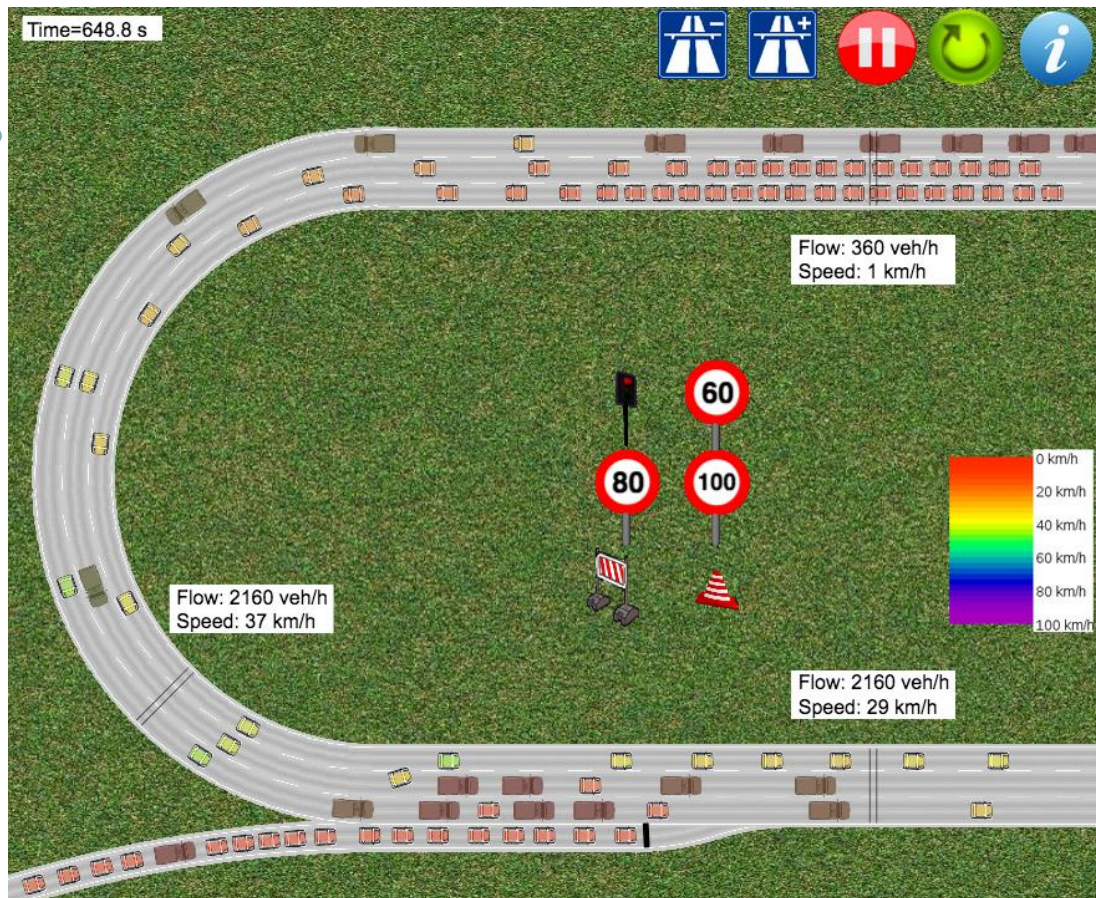




ТЕОРИЯ ГРАФОВ

Потоки в сетях

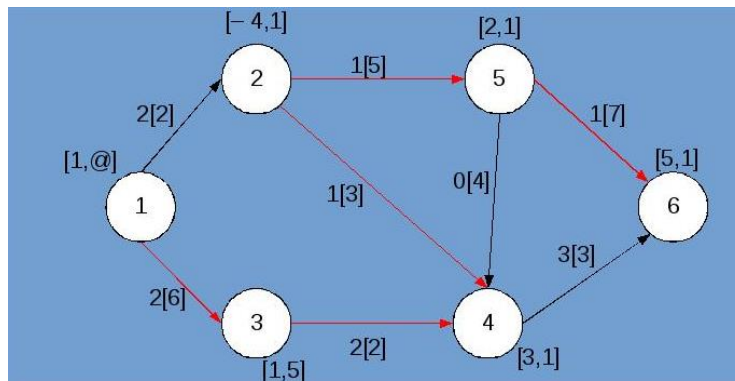
Задачи планирования потока в транспортных сетях

К классу потоковых задач (задачи планирования потока в сетях):

- Имеется система, модель которой может быть представлена в виде ориентированного графа.
- На вход системы поступает *поток* с некоторой интенсивностью λ .
- Пропускная способность элементов системы *ограничена*.
- Требуется так спланировать поток в системе, чтобы ***не нарушались ограничения пропускной способности*** элементов систем, и ***интенсивность потока была максимальной*** (или равной требуемой величине).

Пример практической задачи

- Существует некая сеть автомобильных дорог, по которым можно проехать из пункта А в пункт В. Дороги могут пересекаться в промежуточных пунктах.
- Количество автомобилей, которые могут проехать по каждому отрезку дороги за единицу времени, **ограничено**: определяется такими факторами, как *ширина проезжей части, качество дорожного покрытия, действующие ограничения скорости движения* и т. д. (обычно называют **пропускной способностью дороги**).
- Каково максимальное количество автомобилей, которые могут проехать из пункта А в пункт В за единицу времени без образования пробок на дорогах (обычно называют **автомобильным потоком**)?
- Можно поставить и другой вопрос: какие дороги и насколько необходимо расширить или улучшить, чтобы **увеличить максимальный автомобильный поток на заданную величину**?



Пример практической задачи

- Пусть существует система машин для *производства готовых изделий из сырья и последовательность технологических операций* может быть различной (т. е. операции могут выполняться на разном оборудовании или в разной последовательности), но все ***допустимые варианты заранее строго определены.***
- ***Максимальная производительность*** каждой единицы оборудования заранее известна и постоянна.
- Какова ***максимально возможная производительность всей системы*** в целом?
- Как нужно организовать производство для достижения ***максимальной производительности?***

История задачи

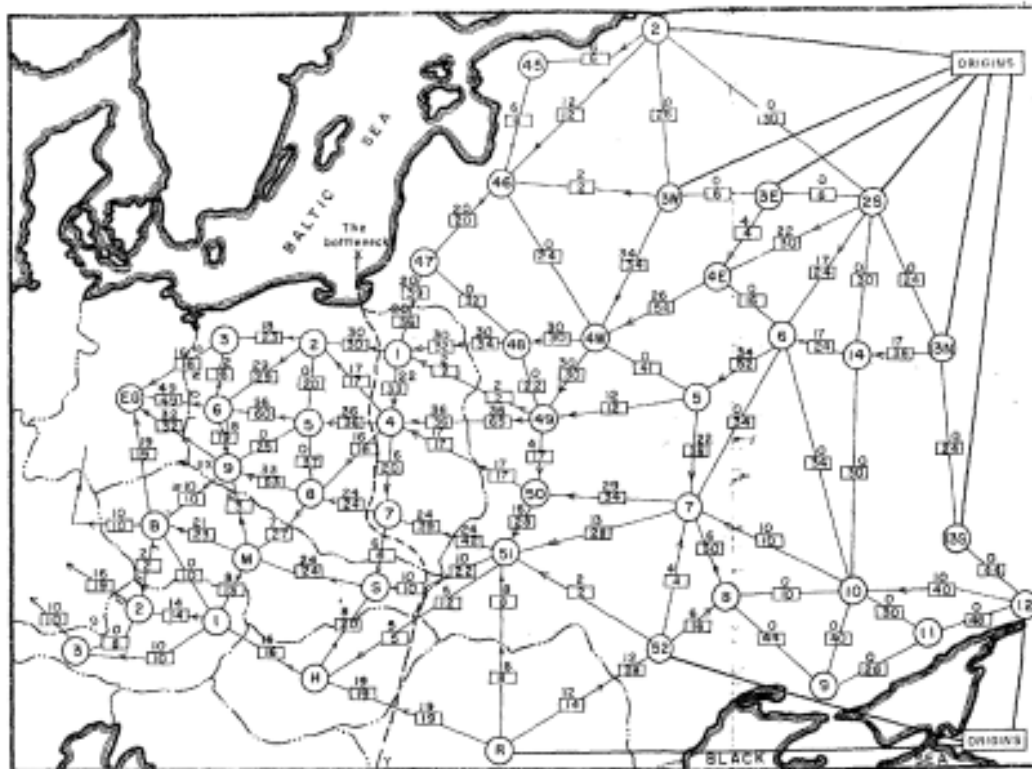


Fig. 7 — Traffic pattern: entire network available

Legend:

— International boundary

⊙ Railway operating division

→ 12 ← Capacity: 12 each way per day. Required flow of 9 per day toward destinations (in direction of arrow) with equivalent number of returning trains in opposite direction

All capacities in trains / 1000's of tons each way per day

Origins: divisions 2, 3A, 3C, 25, 13A, 13B, 12, 52 (USSR), and Roumania

Destinations: divisions 3, 6, 8 (Poland); 8 (Czechoslovakia); and 2, 3 (Austria)

Alternative destinations: Germany or East Germany

Note: 11A of Division 8, Poland

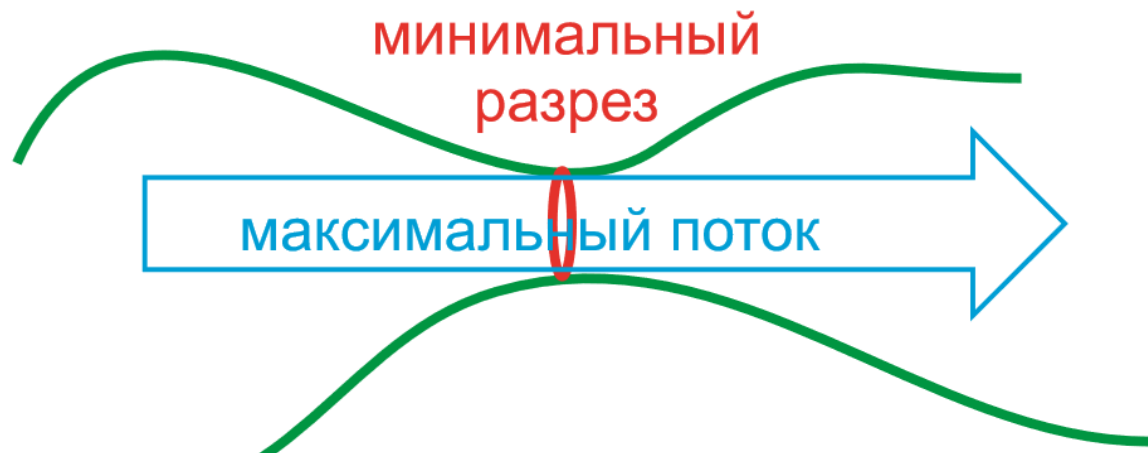
Одно из первых упоминаний задачи о максимальном потоке и минимальном разрезе (середина 1950-х).

История задачи о максимальном потоке

- Задача о максимальном потоке в сети изучается уже более 60 лет. Интерес к ней подогревается огромной практической значимостью этой проблемы. Методы решения задачи применяются на транспортных, коммуникационных, электрических сетях, при моделировании различных процессов физики и химии, в некоторых операциях над матрицами, для решения родственных задач теории графов.....
- Раньше эта задача решалась **симплекс методом линейного программирования**, что было крайне не эффективно. **Форд и Фалкерсон** предложили рассматривать для решения задачи о максимальном потоке ориентированную сеть и искать решение с помощью итерационного алгоритма. В течение 20 последующих лет все передовые достижения в исследовании данной задачи базировались на их методе.
- В 1970 г. Диниц, предложил решать задачу с использованием **вспомогательных бесконтурных сетей и псевдомаксимальных потоков**, что намного увеличило быстродействие разрабатываемых алгоритмов. А в 1974 г. Карзанов улучшил метод Диница, введя такое понятие как **предпоток**. Алгоритмы Диница и Карзанова, как и исследования Форда и Фалкерсона, внесли огромный вклад в решение данной проблемы. На основе их методов 15 лет достигались наилучшие оценки быстродействия алгоритмов.

История задачи о максимальном потоке

- В 1986 г. появился третий метод, который можно отнести к фундаментальным. Этот метод был разработан Голдбергом и Тарьяном, и получил название ***Push-Relabel метода***. Для нахождения максимального потока, он использует *предпотоки и метки*, изменяемые во время работы алгоритма. Push-Relabel алгоритмы очень эффективны, и исследуются до сих пор.
- В 1997 г. Голдберг и Рао предложили алгоритм, *присваивающий дугам неединичную длину*. Это самый современный из всех известных алгоритмов. Асимптотическая оценка его быстродействия превзошла **$O(n*m)$** , о такой скорости многие годы можно было только мечтать.

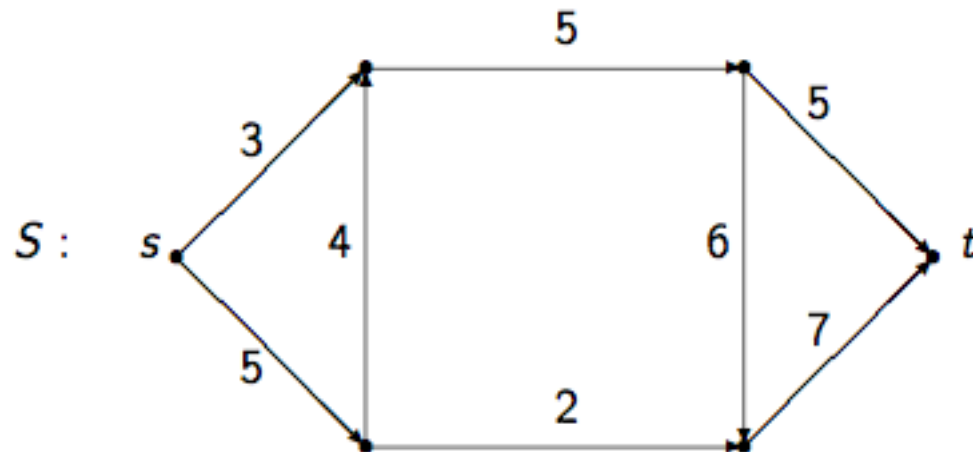


Определение сети

Пусть $G = (V, E)$ — орграф с одним источником $s \in V$ и одним стоком $t \in V$, где $s \neq t$.

Сетью $S = (G; c)$ назовем орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное действительное число $c(e)$, называющееся **пропускной способностью** этой дуги e .

Сеть $S = (G; c)$:



Поток в сети

Потоком в сети $S = (G; c)$, где $G = (V, E)$, называется такая функция $f : E \rightarrow \mathbb{R}_+$, приписывающая каждой дуге $e \in E$ неотрицательное число $f(e)$, называемое **поток** по этой дуге e , для которой выполняются свойства:

- 1) для каждой дуги $e \in E$ верно $f(e) \leq c(e)$, т. е. поток по любой дуге не превышает пропускную способность этой дуги;
- 2) для каждой вершины $v \in V$, $v \neq s, t$, верно

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0,$$

т. е. поток, входящий в любую вершину, кроме источника и стока, без остатка выходит из нее (сохранение потока).

При этом неотрицательное число $p_f = \sum_{e \in A(s)} f(e)$ называется **величиной потока f** .

Число $w(f) \stackrel{\text{Def}}{=} \text{div}(f, s)$ называется **величиной потока f** . Если $f(u, v) = c(u, v)$, то дуга (u, v) называется **насыщенной**.

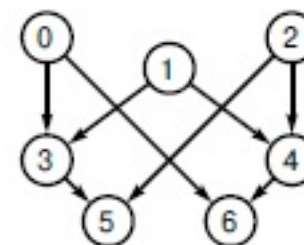
Фиктивные источники и стоки

Максимальный поток в сетях общего вида.

- Нужно найти поток в сети, который максимизирует суммарный отток из ее истоков (и, следовательно, приток в ее стоки).
- По соглашению в сети, в которой нет истоков и/или стоков, поток считается нулевым.

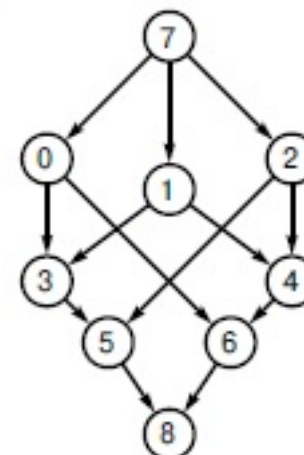
пропускная
способность

0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3



пропускная
способность

0-3	2
0-6	3
1-3	3
1-4	1
2-4	1
2-5	1
3-5	2
4-6	3
7-0	5
7-1	4
7-2	2
5-8	3
6-8	6

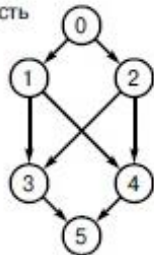


Разновидность задачи

- Задача о максимальном потоке в сетях общего вида эквивалентна задаче о максимальном потоке в st -сетях.
- Задача вычисления максимального потока в транспортной сети с ограничениями на пропускную способность вершин эквивалентна стандартной задаче о максимальном потоке.
- Задача вычисления максимального потока в ациклических сетях эквивалентна стандартной задаче о максимальном потоке.
- Задача о максимальном потоке для неориентированных st -сетей сводится к задаче о максимальном потоке для st -сетей.
- Задача о допустимом потоке сводится к задаче о максимальном потоке.

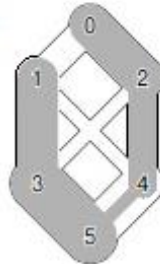
пропускная
способность

0-1	2
0-2	3
1-3	3
1-4	1
2-3	1
2-4	2
3-5	4
4-5	3



поток

0-1	0
0-2	3
1-3	3
1-4	0
2-3	0
2-4	2
3-5	4
4-5	1



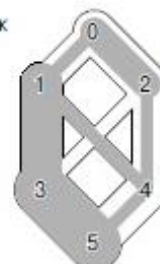
поток

0-1	0
0-2	3
1-3	2
1-4	1
2-3	0
2-4	2
3-5	3
4-5	2



поток

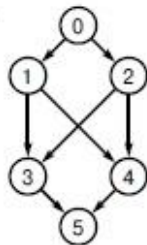
0-1	1
0-2	2
1-3	3
1-4	1
2-3	0
2-4	1
3-5	4
4-5	1



	0	1	2	3	4	5
запас	3	3		1		
предложение			1		1	5

пропускная
способность

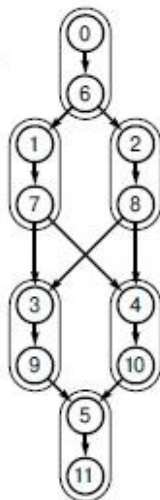
0-1	2
0-2	3
1-3	3
1-4	1
2-3	1
2-4	1
3-5	2
4-5	3



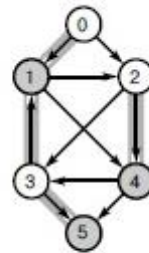
	0	1	2	3	4	5
capV	4	4	3	1	3	4

пропускная
способность

6-1	2
6-2	3
7-3	3
7-4	1
8-3	1
8-4	1
9-5	2
10-5	3
0-6	4
1-7	4
2-8	3
3-9	1
4-10	3
5-11	4

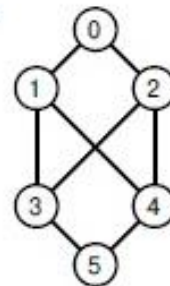


0-1	2
0-2	3
1-2	3
1-4	2
2-3	2
2-4	1
3-1	3
3-5	2
4-3	3
4-5	3



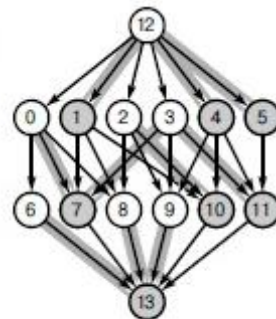
пропускная
способность

0-1	2
0-2	3
1-3	3
1-4	1
2-3	1
2-4	1
3-5	2
4-5	3



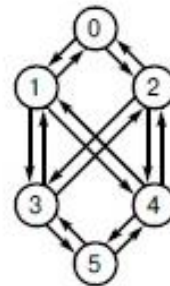
0-6	25	12-0	5	6-13	5
1-7	25	12-1	5	7-13	5
2-8	25	12-2	3	8-13	3
3-9	25	12-3	5	9-13	5
4-10	25	12-4	6	10-13	6
5-11	25	12-5	0	11-13	0

0-7	2
0-8	3
1-8	3
1-10	2
2-9	2
2-10	1
3-7	3
3-11	2
4-9	3
4-11	3



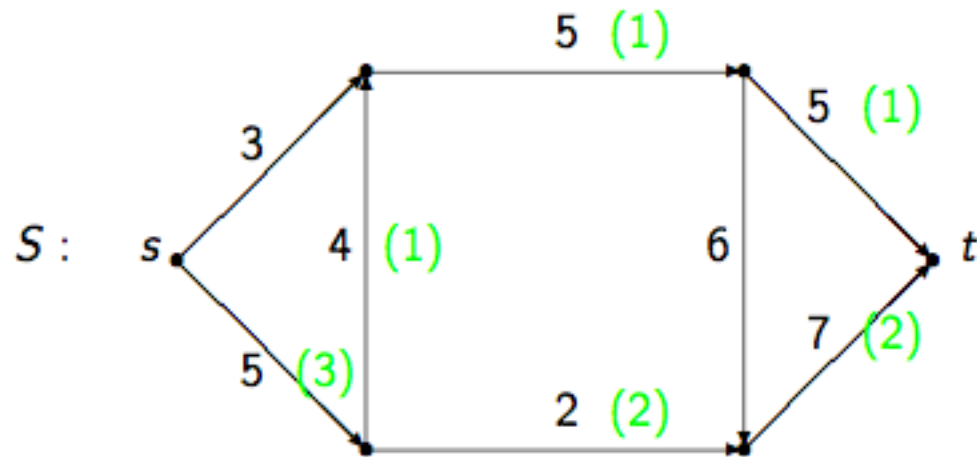
пропускная
способность

0-1	2
1-0	2
0-2	3
2-0	3
1-3	3
3-1	3
1-4	1
4-1	1
2-3	1
3-2	1
2-4	1
4-2	1
3-5	2
5-3	2
4-5	3
5-4	3



Примеры потоков в сетях

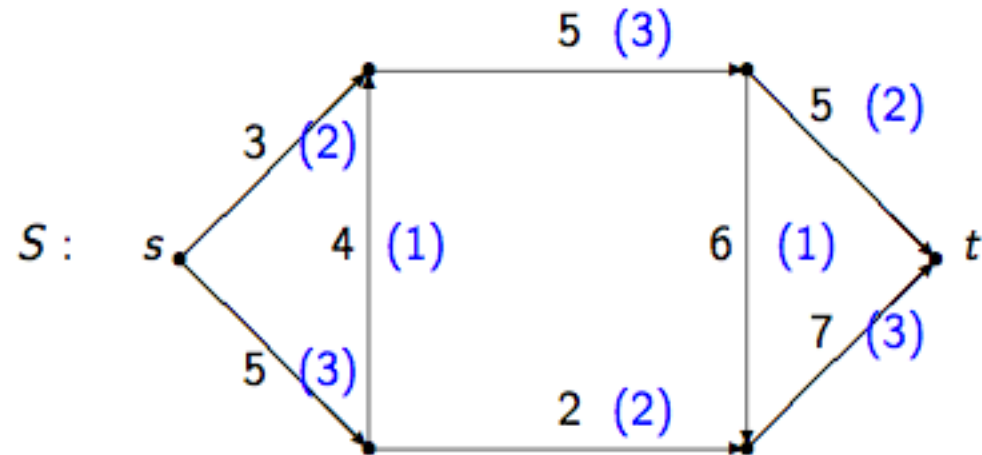
Поток f_1 в сети $S = (G; c)$:



Получаем: $p_{f_1} = 3$.

Примеры потоков в сетях

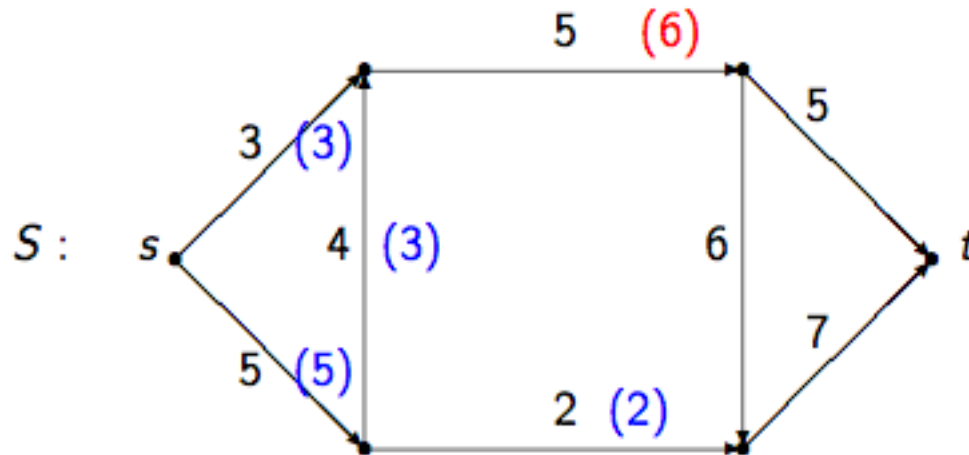
Можно построить другой поток f_2 в этой же сети $S = (G; c)$:



Получаем: $p_{f_2} = 5$.

Примеры потоков в сетях

Попытаемся построить поток f_3 в этой же сети $S = (G; c)$:



Поток f_3 с $p_{f_3} = 8$ построить не удалось.

Разрезы

Пусть $S = (G; c)$ — сеть, где $G = (V, E)$.

Если $X \subseteq V$, $s \in X$, $t \notin X$ и $Y = V \setminus X$, то

$$R = \{e \in E \mid e = (v, w), v \in X, w \in Y\},$$

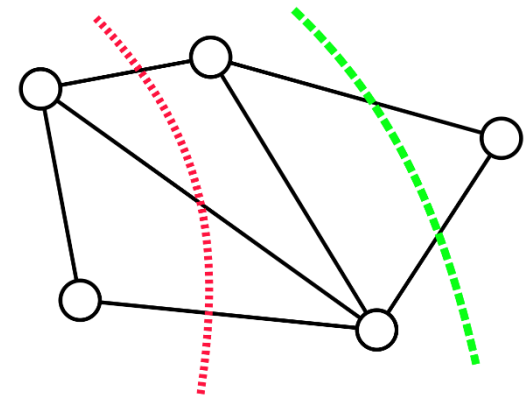
называется **разрезом** сети S и обозначается $R = (X, Y)$.

Если удалить из сети S все дуги разреза R , то останется сеть, в которой нет ни одного направленного пути из s в t .

Пропускной способностью разреза $R(X, Y)$ называется неотрицательное число

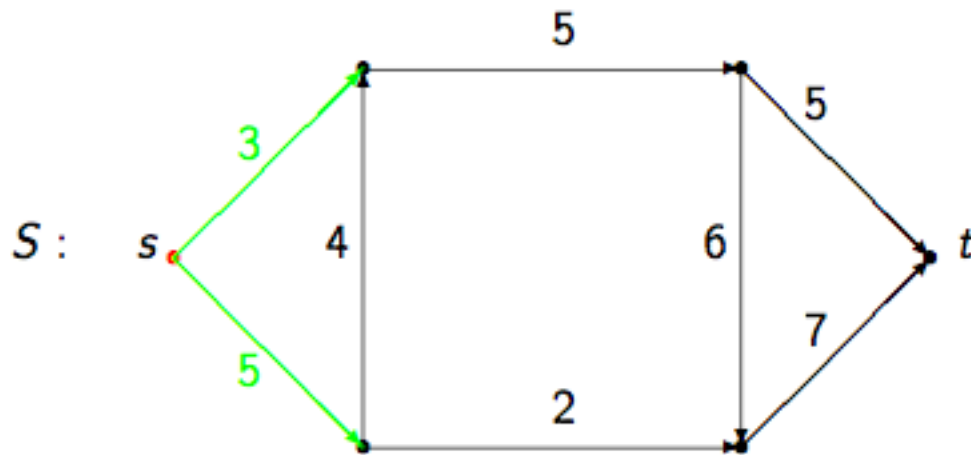
$$c(R) = \sum_{e \in R} c(e),$$

т. е. равное сумме пропускных способностей его дуг.



Примеры разрезом

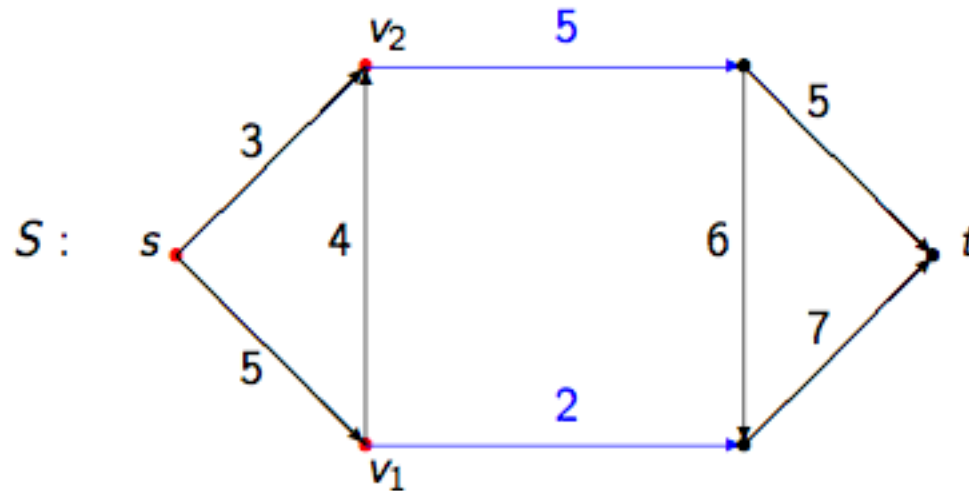
Рассмотрим разрез $R_1 = (X_1, Y_1)$ в сети S , где $X_1 = \{s\}$:



Получаем: $c(R_1) = 5 + 3 = 8$.

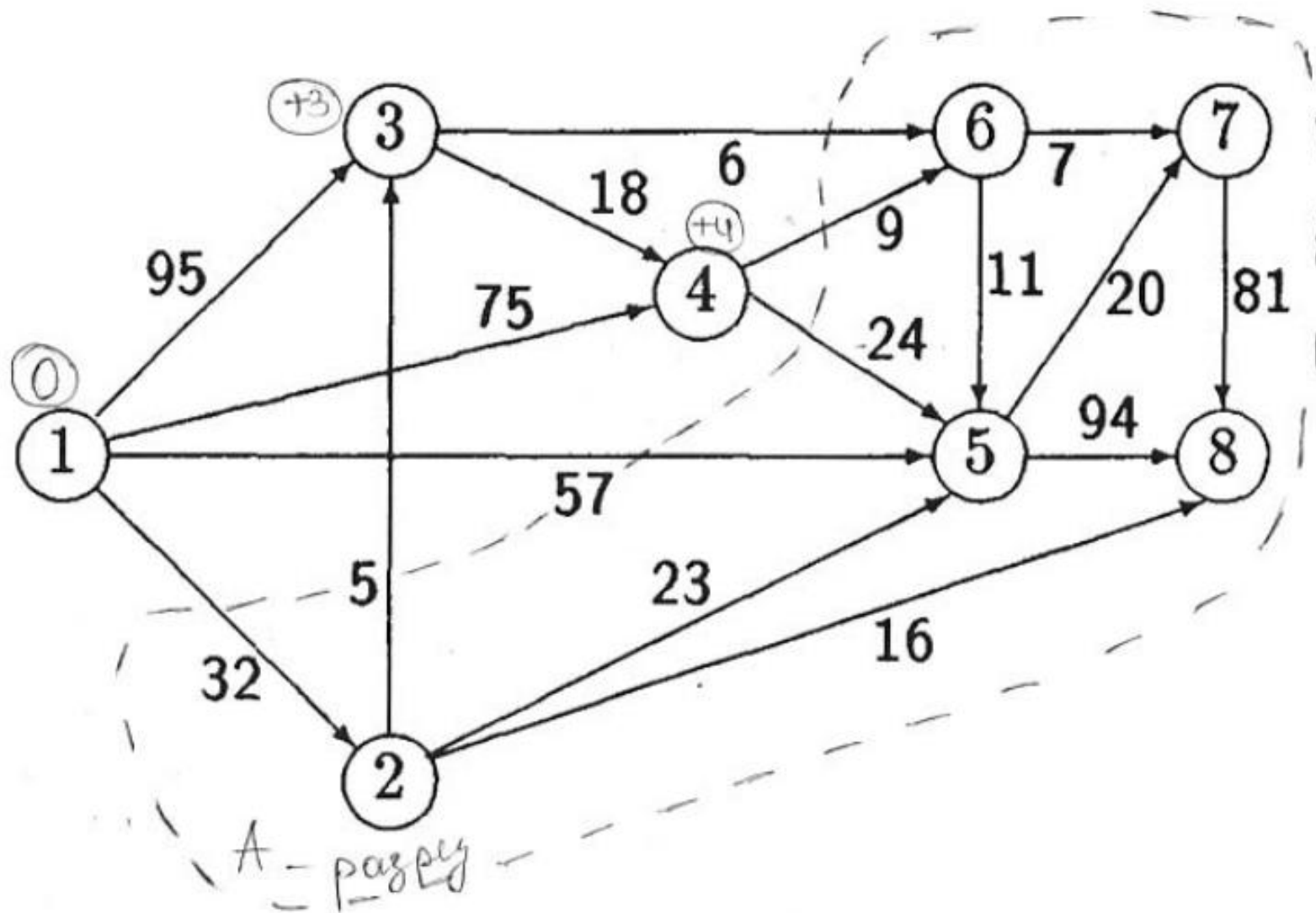
Примеры разрезом

Рассмотрим разрез $R_2 = (X_2, Y_2)$ в сети S , где $X_2 = \{s, v_1, v_2\}$:



Получаем: $c(R_2) = 2 + 5 = 7$.

Примеры разрезом



Лемма о величине потока в сети

Лемма 1 (о величине потока в сети). Пусть $S = (G; c)$ — сеть, где $G = (V, E)$. Величина любого потока в сети S не превосходит пропускной способности любого ее разреза.

Доказательство. Пусть $f : E \rightarrow \mathbb{R}_+$ — произвольный поток в сети S , а $R = (X, Y)$ — произвольный ее разрез, где $X \subseteq V$, $s \in X$, $t \notin X$, $Y = V \setminus X$.

$$p_f = \sum_{e \in A(s)} f(e) - \sum_{e \in B(s) = \emptyset} f(e).$$

В промежуточных вершинах поток не остается, поэтому эту сумму можно добавить все вершины из X ($t \notin X$):

$$\begin{aligned} p_f &= \sum_{v \in X} \left(\sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) \right) = \\ &= \sum_{v \in X, (v, w) \in A(v)} f(v, w) - \sum_{v \in X, (w, v) \in B(v)} f(w, v). \end{aligned}$$

Доказательство

Доказательство. Если $v, w \in X$, то $f(v, w)$ в полученной сумме встречается и со знаком «+», и со знаком «-». Поэтому ненулевой вклад вносят потоки по дугам, в которых одна из вершин лежит в X , а другая вершина — в Y . Значит,

$$\begin{aligned} p_f &= \sum_{v \in X, w \in Y, (v, w) \in E} f(v, w) - \sum_{v \in X, w \in Y, (w, v) \in E} f(w, v) \leq \\ &\leq \sum_{v \in X, w \in Y, (v, w) \in E} f(v, w) \leq \sum_{v \in X, w \in Y, (v, w) \in E} c(v, w) = c(R). \end{aligned}$$

Поэтому $p_f \leq c(R)$.



ЛЕММА 1 $w(f) = F(P^+) - F(P^-)$.

ДОКАЗАТЕЛЬСТВО Рассмотрим сумму $W := \sum_{v \in S} \text{div}(f, v)$. Пусть дуга $(u, v) \in E$. Если $u, v \in S$, то в сумму W попадают два слагаемых для этой дуги: $+f(u, v)$ при вычислении $\text{div}(f, u)$ и $-f(u, v)$ при вычислении $\text{div}(f, v)$, итого 0. Если $u \in S$, $v \in T$, то в сумму W попадает одно слагаемое $+f(u, v)$, все такие слагаемые дают $F(P^+)$. Если $u \in T$, $v \in S$, то в сумму W попадает одно слагаемое $-f(u, v)$, все такие слагаемые дают $F(P^-)$. Таким образом, $W = F(P^+) - F(P^-)$. С другой стороны, $W = \sum_{v \in S} \text{div}(f, v) = \text{div}(f, s) = w(f)$. $\square$

ЛЕММА 2 $\text{div}(f, s) = -\text{div}(f, t)$.

ДОКАЗАТЕЛЬСТВО Рассмотрим разрез $P := (S, T)$, где $S := V - t$, а $T := \{t\}$. Имеем:
$$\text{div}(f, s) = w(f) = F(P^+) - F(P^-) = F(P^+) = \sum f(v, t) = -\text{div}(f, t). \quad \square$$

ЛЕММА 3 $w(f) \leq F(P)$.

ДОКАЗАТЕЛЬСТВО $w(f) = F(P^+) - F(P^-) \leq F(P^+) \leq F(P)$. $\square$

ЛЕММА 4 $\max_f w(f) \leq \min_P C(P)$.

ДОКАЗАТЕЛЬСТВО По предыдущей лемме $w(f) \leq F(P)$, следовательно, $\max_f w(f) \leq \min_P F(P)$. По определению $F(P) \leq C(P)$, следовательно, $\min_P F(P) \leq \min_P C(P)$. Имеем: $\max_f w(f) \leq \min_P C(P)$. $\square$

Теорема Форда-Фалкерсона

Теорема 1 (Л. Р. Форд, Д. Р. Фалкерсон, 1962). Пусть $S = (G; c)$ — сеть, где $G = (V, E)$. Величина максимального потока $p_{\max}$ в сети S совпадает с минимальной пропускной способностью $r_{\min}$ ее разрезом.

Доказательство. Пусть $R_{\min} = (X, Y)$ — разрез сети S с минимальной пропускной способностью $c(R_{\min}) = r_{\min}$ и $f : E \rightarrow \mathbb{R}_+$ — поток в сети S с максимальной величиной $p_f = p_{\max}$.

По лемме 1 выполняется $p_f \leq c(R_{\min})$.

Покажем, что $p_f = c(R_{\min})$.

По индукции определим множество вершин $X \subseteq V$.

Базис индукции: $s \in X$.

Индуктивный переход:

- 1) если $v \in X$, $(v, w) \in E$ и $f(v, w) < c(v, w)$, то $w \in X$;
- 2) если $v \in X$, $(w, v) \in E$ и $f(w, v) > 0$, то $w \in X$.

Т. е. X — множество вершин, в которые можно перейти из источника s по прямым дугам с **ненасыщенным** потоком (т. е. который меньше пропускной способности этой дуги) и по обратным дугам с **ненулевым** потоком. Положим $Y = V \setminus X$.

Доказательство

Доказательство. 1. Если $t \notin X$, то $R = (X, Y)$ — разрез. Тогда по доказательству леммы 1:

$$p_f = \sum_{v \in X, w \in Y, (v, w) \in E} f(v, w) - \sum_{v \in X, w \in Y, (w, v) \in E} f(w, v).$$

Но если $v \in X$, $w \in Y$ и $(v, w) \in E$, то $f(v, w) = c(v, w)$, т. к. иначе вершину w добавили бы в X .

Также если $v \in X$, $w \in Y$ и $(w, v) \in E$, то $f(w, v) = 0$, т. к. иначе вершину w добавили бы в X .

Доказательство. Поэтому

$$= \sum_{v \in X, w \in Y, (v, w) \in E} c(v, w) = c(R).$$

Из $c(R_{\min}) \leq c(R)$ и $p_f \leq c(R_{\min})$ получаем $p_f = c(R_{\min})$.

Доказательство

Доказательство. 2. Если $t \in X$, то в S найдется путь P (**без учета направлений дуг**) из s в t , состоящий только из прямых дуг с ненасыщенным потоком и из обратных дуг с ненулевым потоком:

$$P = se_1 v_1 e_2 v_2 \dots v_{m-1} e_m t,$$

где для каждого $i = 1, \dots, m$ выполняется:

1) если $e_i = (v_{i-1}, v_i) \in E$, то $f(e_i) < c(e_i)$;

2) если $e_i = (v_i, v_{i-1}) \in E$, то $f(e_i) > 0$.

Доказательство. Положим:

$$a = \min_{e_j = (v_{j-1}, v_j) \in E} c(e_j) - f(e_j);$$

$$b = \min_{e_j = (v_j, v_{j-1}) \in E} f(e_j);$$

$$d = \min(a, b).$$

Отметим, что $d > 0$.

Доказательство

Доказательство. Теперь построим новый поток f' в сети S , изменив потоки по дугам пути P : для каждого $j = 1, \dots, m$ положим:

- 1) если $e_j = (v_{j-1}, v_j) \in E$, то $f'(e_j) = f(e_j) + d$;
- 2) если $e_j = (v_j, v_{j-1}) \in E$, то $f'(e_j) = f(e_j) - d$.

Получаем: $p_{f'} = p_f + d > p_{\max}$, чего не может быть. Значит, случай 2 не возможен, выполняется случай 1 и

$$p_f = p_{\max} = c(R_{\min}).$$



Сложность:

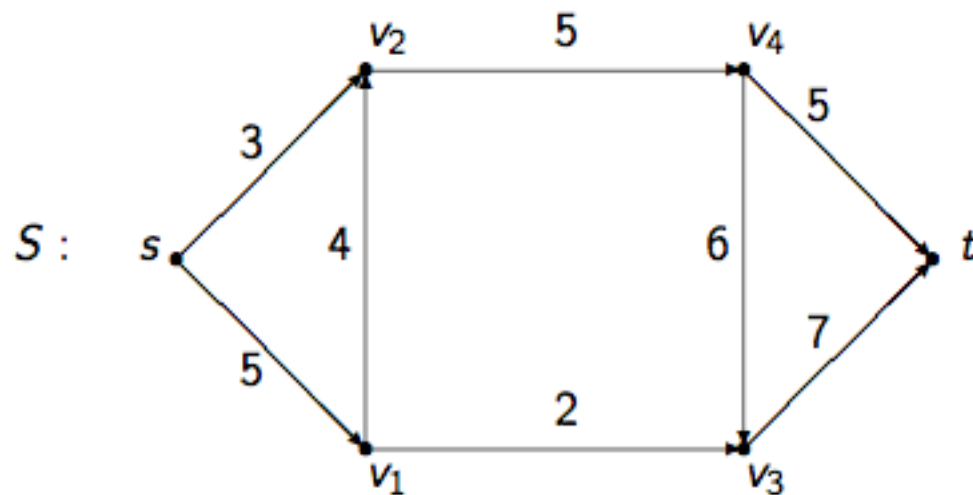
$$O(p \cdot q \cdot U)$$

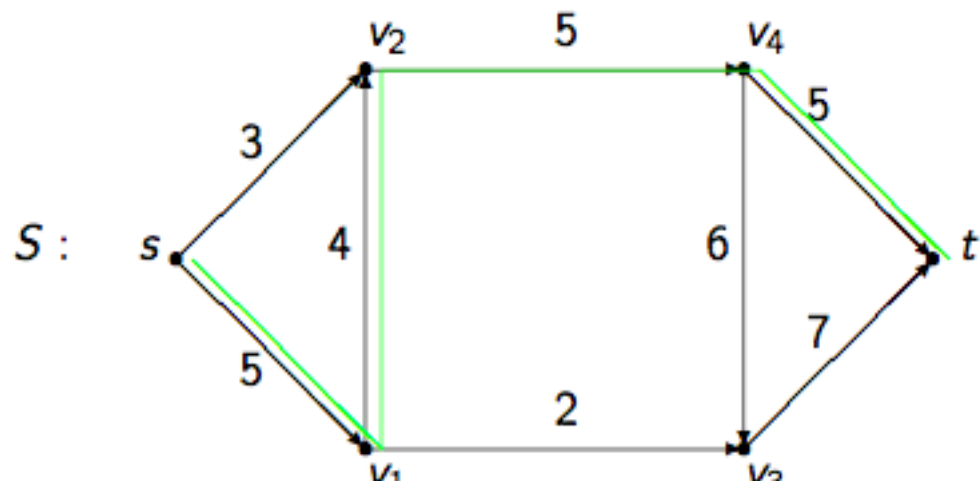
U – наибольшая величина максимальной пропускной способности сети (оценки, содержащие U , являются слабо полиномиальными), p – количество вершин, q – количество ребер.

Алгоритм Форда-Фалкерсона (увеличивающего пути)

По доказательству теоремы 1 можно описать алгоритм построения максимального потока в сети. Он называется **алгоритмом расстановки пометок**.

Рассмотрим сеть S , и пусть f_0 — нулевой поток.

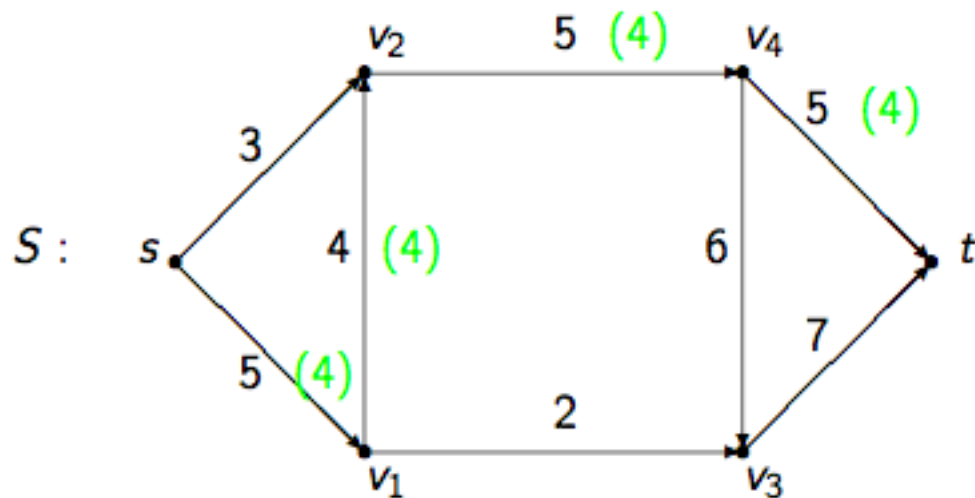


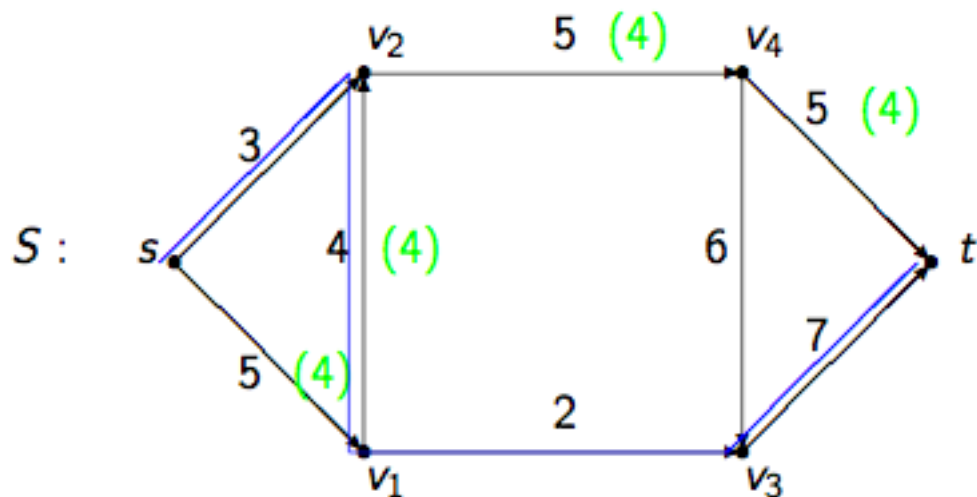


Нашли путь:

$$P_1 = s(s, v_1)v_1(v_1, v_2)v_2(v_2, v_4)v_4(v_4, t)t.$$

Получаем: $d = a = \min(5, 4, 5, 5) = 4$, и поток f_0 можно увеличить до нового потока f_1 , $p_{f_1} = 4$.

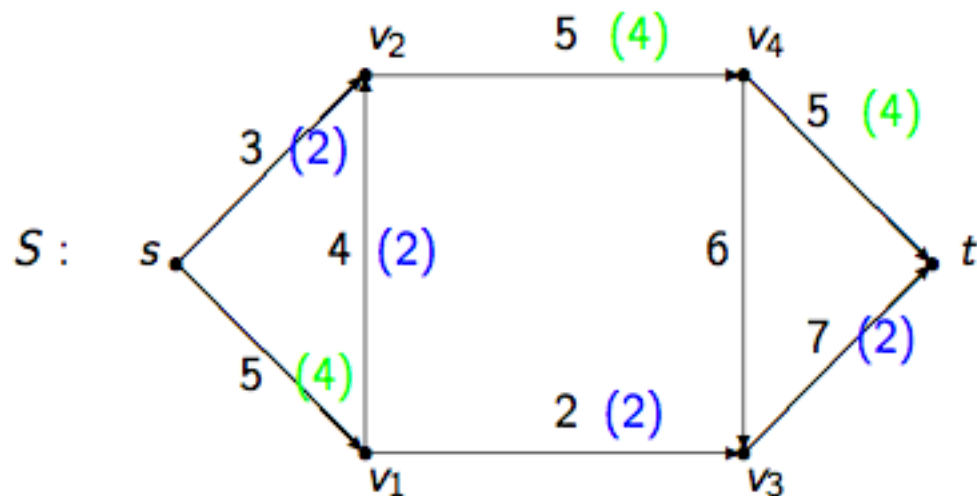


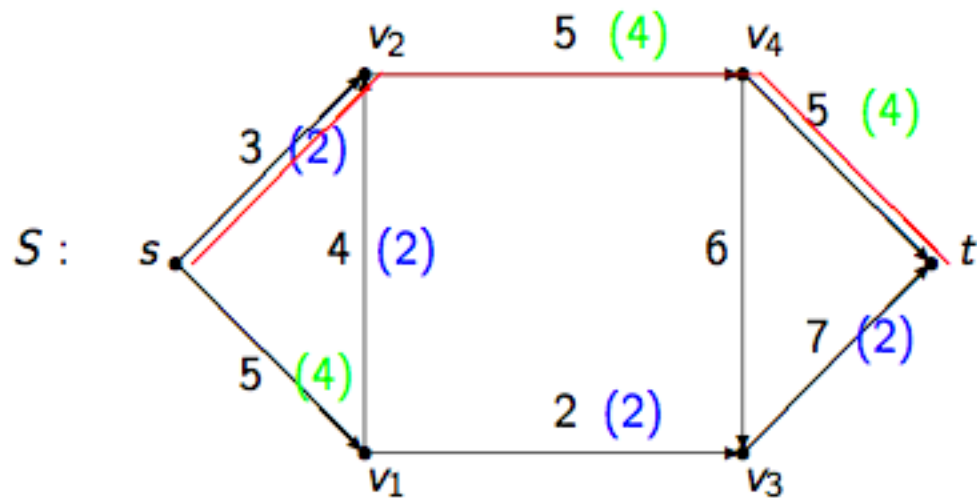


Нашли новый путь:

$$P_2 = s(s, v_2)v_2(v_1, v_2)v_1(v_1, v_3)v_3(v_3, t)t.$$

Получаем: $a = \min(3, 2, 7) = 2$, $b = \min(4) = 4$,
 $d = \min(2, 4) = 2$, и поток f_1 можно увеличить до нового
 потока f_2 , $p_{f_2} = 4 + 2 = 6$.

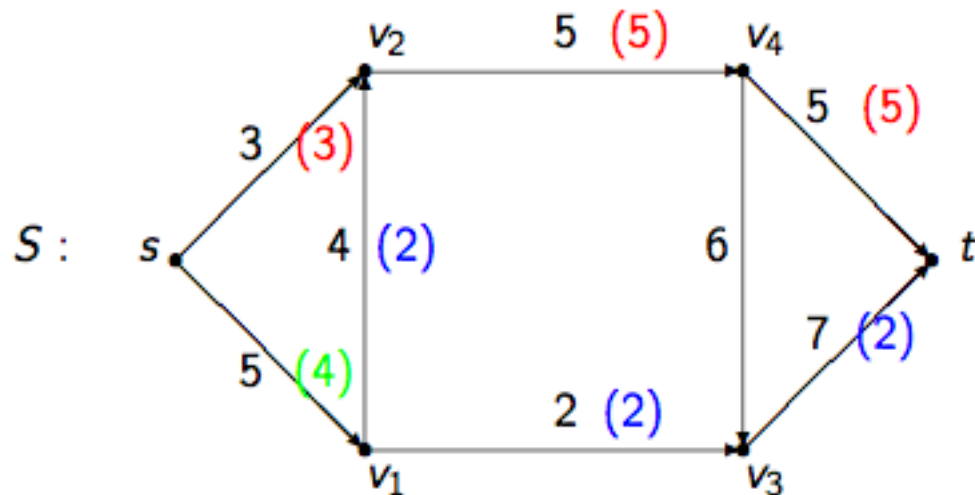




Нашли еще один путь:

$$P_3 = s(s, v_2)v_2(v_2, v_4)v_4(v_4, t)t.$$

Получаем: $d = a = \min(1, 1, 1) = 1$, и поток f_2 можно увеличить до нового потока f_3 , $p_{f_3} = 6 + 1 = 7$.



Псевдо-код алгоритма

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей $C : \text{array } [1..p, 1..p] \text{ of real}$.

Выход: матрица максимального потока $F : \text{array } [1..p, 1..p] \text{ of real}$.

for $u, v \in V$ **do**

$F[u, v] := 0$ { вначале поток нулевой }

end for

$M : \{ \text{итерация увеличения потока} \}$

for $v \in V$ **do**

$S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$ { инициализация }

end for

$S[s] := 1; P[s] := (+, \text{nil}, \infty)$ { так как $s \in S$ }

repeat

$a := 0$ { признак расширения S }

for $v \in V$ **do**

if $S[v] = 1 \ \& \ N[v] = 0$ **then**

for $u \in \Gamma(v)$ **do**

if $S[u] = 0 \ \& \ F[v, u] < C[v, u]$ **then**

$S[u] := 1; P[u] := (+, v, \min(P[v].\delta, C[v, u] - F[v, u])); a := 1$

end if

```

end for
for  $u \in \Gamma^{-1}(v)$  do
    if  $S[u] = 0 \ \& \ F[u, v] > 0$  then
         $S[u] := 1; P[u] := (-, v, \min(P[v].\delta, F[u, v])); a := 1$ 
    end if
end for
 $N[v] := 1$  { узел  $v$  отмечен }
end if
end for
if  $S[t]$  then
     $x := t$  { текущий узел аугментальной цепи }
     $\delta := P[t].\delta$  { величина изменения потока }
    while  $x \neq s$  do
        if  $P[x].s = +$  then
             $F[P[x].n, x] := F[P[x].n, x] + \delta$  { увеличение потока }
        else
             $F[x, P[x].n] := F[x, P[x].n] - \delta$  { увеличение потока }
        end if
         $x := P[x].n$  { предыдущий узел аугментальной цепи }
    end while
    goto  $M$ 
end if
until  $a = 0$ 

```

Этап 1. Путь $s \xrightarrow{12} x_1 \xrightarrow{14} x_3 \xrightarrow{15} t$. $\delta = \min(12, 14, 15) = 12$. Увеличим по этому пути поток до 12 единиц, ребро (s, x_1) становится насыщенным. Поставим величину потока на дугах (x_1, x_3) и (x_3, t) .

Путь $s \xrightarrow{13} x_3 \xrightarrow{12(15)} t$. $\delta = \min(13, 15 - 12) = 3$. Поток можно увеличить на три единицы. Дуга (x_3, t) станет насыщенной. Путь $s \xrightarrow{3(13)} x_3 \xrightarrow{7} x_4 \xrightarrow{8} x_2 \xrightarrow{8} t$. Можно увеличить поток на семь единиц; дуга (x_3, x_4) станет насыщенной, потоки на дугах примут вид как на рис. 3.69: $s \xrightarrow{10(13)} x_3 \xrightarrow{7(7)} x_4 \xrightarrow{7(8)} x_2 \xrightarrow{7(8)} t$. Больше путей нет. Конец первого этапа.

Этап 2. Рассмотрим теперь маршруты, содержащие противоположные дуги. Маршрут $s \xrightarrow{10(13)} x_3 \xleftarrow{12(14)} x_1 \xrightarrow{11} x_2 \xrightarrow{7(8)} t$. Поток можно увеличить на единицу на дуге (x_2, t) . Тогда потоки по дугам этого маршрута станут такими $s \xrightarrow{11(13)} x_3 \xleftarrow{11(14)} x_1 \xrightarrow{1(11)} x_2 \xrightarrow{8(8)} t$. Дуга (x_2, t) стала насыщенной.

Больше маршрутов нет. Поток максимален. Делаем разрез вокруг t по насыщенным дугам и получаем его величину $15 + 8 = 23$ единицы.

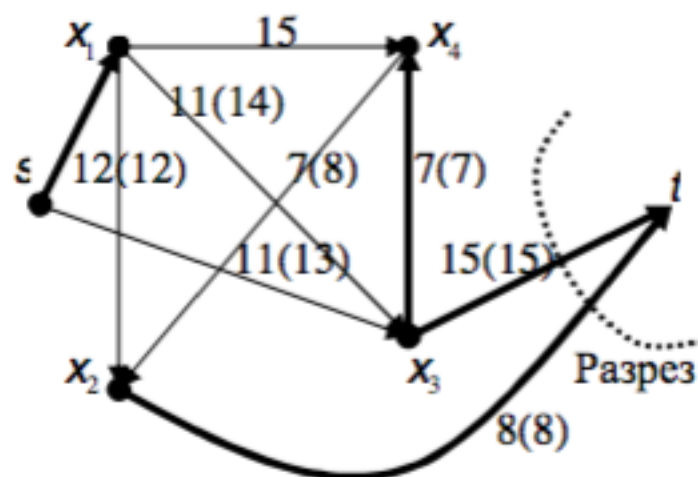
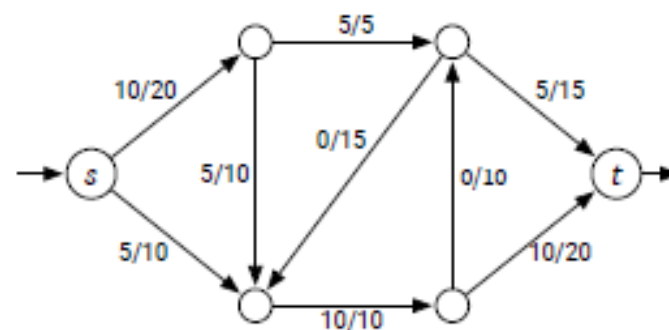
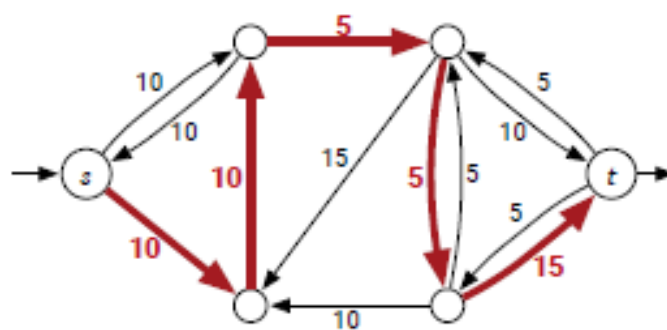
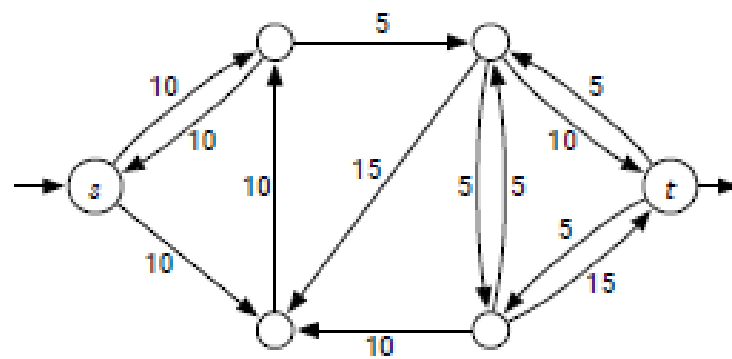
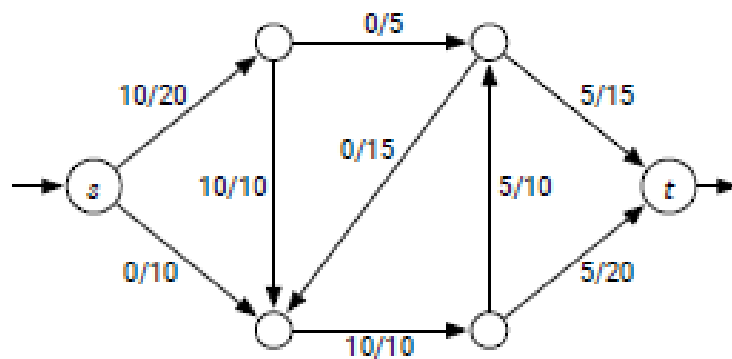


Рис. 3.69

Пример



Примеры решаемых задач

- **Задача о максимальном паросочетании.** На вход дается N — количество мальчиков, M — количество девочек и список, какой мальчик с какой из девочек хочет танцевать (таких может быть несколько). Надо определить *максимальное количество одновременно танцующих пар*.
- **Испорченный паркет.** У паркета $N \times M$, некоторые клетки могут быть испорчены. Их необходимо закрыть новыми плитками. Плитки бывают размером 2×1 (можно поворачивать, но нельзя разрезать) ценой A , и 1×1 ценой B . Спрашивается, какую *минимальную сумму нужно потратить*, что бы заложить испорченные плитки паркета. Естественно, новые плитки не должны перекрывать никакие другие плитки.
- **Задача живопись.** Дана матрица $N \times M$ с клетками, покрашенными либо в черный, либо в белый цвета. W — цена перекраски черного квадрата в белый, B — белого в черный. После перекраски, между всеми соседними квадратами разных цветов нужно провести серую линию, ценой G . Надо так *оптимально перекрасить матрицу* (или ничего не делать), что бы *потратить минимальную сумму*.

Примеры решаемых задач

- **Задача с ограничением на вершины.** Пусть надо найти величину максимального потока и на вершины наложено ограничение, сколько они могут пропустить.
- **Минимальный разрез.** Дан граф. Сколько вершин надо удалить, что бы не существовало пути из A в B ?
- **Коллекционирование монет.** Есть n коллекционеров и m видов монет. Для вступления в клуб, необходимо иметь не меньше одной монеты каждого типа. Вы (у Вас номер 1) можете меняться с коллекционерами имеющимися монетами. Любой коллекционер обменяет монету свою монету a на Вашу монету b , если у него **больше** одной монеты типа a и нету ни одной монеты типа b . Вы, в свою очередь, можете нарушать это правило. Нужно набрать *как можно больше типов монет* по известной ситуации у всех коллекционеров.

Теорема Менгера

- **ТЕОРЕМА** (Менгера). Пусть u и v — несмежные вершины в графе G . Наименьшее число вершин в множестве, разделяющем u и v , равно наибольшему числу вершинно непересекающихся простых $\langle u, v \rangle$ -цепей:

$$\max |P(u, v)| = \min |S(u, v)|.$$

$$\forall P \ (\forall S \ (|P| \leq |S|)).$$

- **ТЕОРЕМА** Для любых двух несмежных вершин u и v графа G наибольшее число рёберно-непересекающихся $\langle u, v \rangle$ -цепей равно наименьшему числу рёбер в $\langle u, v \rangle$ -разрезе.
- **ТЕОРЕМА** Чтобы граф G был n -связным, необходимо и достаточно, чтобы любые две несмежные вершины были соединены не менее чем n вершинно-непересекающимися простыми цепями.

В любом графе G любые две несмежные вершины соединены не менее чем $\chi(G)$ вершинно-непересекающимися простыми цепями.

Связь между теоремой Менгера и теоремой Форда-Фалкерсона

Теорема Менгера (ТМ) является фундаментальной теоремой.

Теорема Форда-Фалкерсона (ТФФ) может быть также получена из нее. Например, ТМ в ориентированной реберной форме является ТФФ при единичной пропускной способности.

Тогда, если «перевести» ребра в мультиребра (т.е. рассматривать мультиграф), то получатся **натуральные числа**, представляющие пропускные способности дуг.

Если пропускные способности – рациональные числа, то можно привести к общему знаменателю и свести к предыдущему случаю. Для вещественных значений можно перейти к пределу в условиях предыдущего случая.

Алгоритмы потокового программирования

- Алгоритм Форда – Фалкерсона (1956, 1962) $O(nmU)$
- Алгоритм кратчайших увеличивающих цепей Эдмондса – Карна (1970) $O(nm^2)$
- Алгоритм локально-максимального увеличения Эдмондса – Карна (1972) $O(m^2 \log U)$
- Алгоритм Диница (1970) $O(n^2 m)$
- Алгоритм Карзанова (1974) $O(n^3)$
- Алгоритм Малхотри – Кумара – Махешвари (1978) $O(n^3)$
- Алгоритм Галила – Наамада (1978) $O(nm \log^2 n)$
- Алгоритм Слейтора – Тарьяна (1980) $O(nm \log n)$
- Алгоритм Голдберга – Тарьяна (1986) $O(nm \log(n^2/m))$
- Алгоритм Черияна – Хагерана – Мелхорна (1989) $E(nm + n^2 \log^2 n)$
- Алгоритм Кинга (1992) $O(nm + n^{2+\epsilon})$
- Алгоритм Голдберга – Рао $O(\min\{n^{2/3}, n^{1/2}\} m \log(n^2/m) \log U)$

U – наибольшая величина максимальной пропускной способности сети (оценки, содержащие U , являются слабо полиномиальными); под $E()$ понимается высокая вероятность такой оценки (для алгоритмов, допускающих случайное поведение); n – количество вершин, m – количество ребер.

Задачи потокового программирования

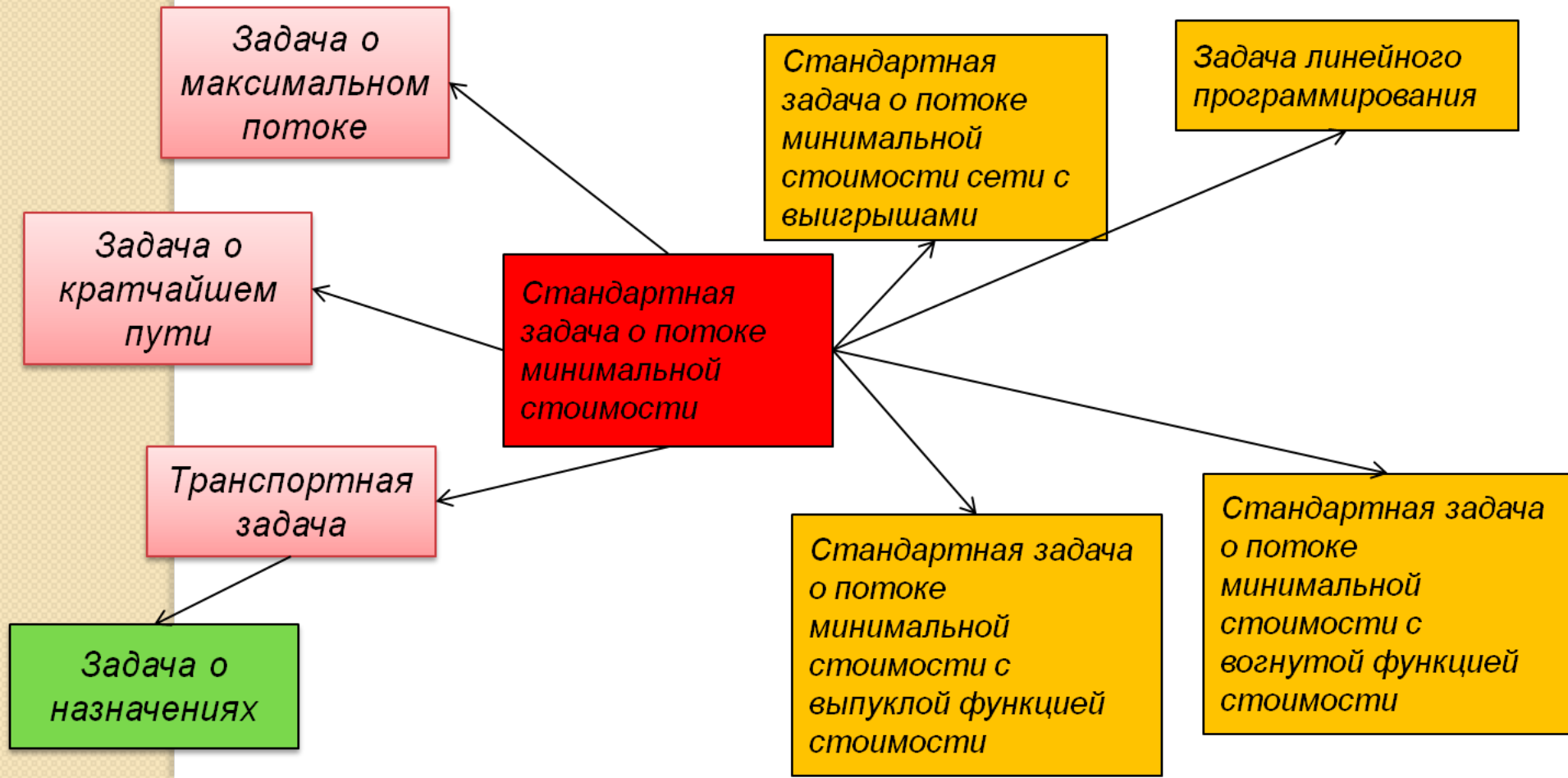
- В задачах потокового программирования поток по каждой дуге является управляемой переменной, а цель состоит в получении **оптимального значения некоторой меры эффективности** за счет выбора соответствующих потоков по каждой дуге сети.
- **МОДЕЛЬ ПОТОКА СЕТИ.** По каждой дуге сети в направлении, указанном стрелкой, протекает некоторый поток. В данной модели каждая дуга характеризуется тремя параметрами: **минимальным значением потока**, который может протекать по дуге(нижняя граница); **пропускной способностью**, которая показывает, какой максимальный поток можно передавать по дуге(верхняя граница); **стоимостью передачи единицы потока** по данной дуге.
- Потоки и пропускные способности обычно измеряются в единицах **интенсивности потока**. Если нижняя граница не указана, то предполагают, что она равна нулю.

Задача оптимизации

- Для каждого узла сети задаются значения потоков, которые по условиям задачи должны входить в сеть или покидать ее в данном узле. Такие потоки называются **внешними** и рассматриваются как некоторые параметры узлов.
- *Положительный внешний поток* входит в сеть в соответствующем узле, а *отрицательный* покидает сеть. Поток **сохраняется в каждом узле**, т. е. сумма потока входящего в некоторый узел по дугам сети, и внешнего потока в узле должна быть равна потоку, выходящему из узла по дугам сети.
- Задача оптимизации состоит в **минимизации общей стоимости передачи потока** при условии, что потоки по дугам удовлетворяют всем упомянутым условиям и ограничениям.

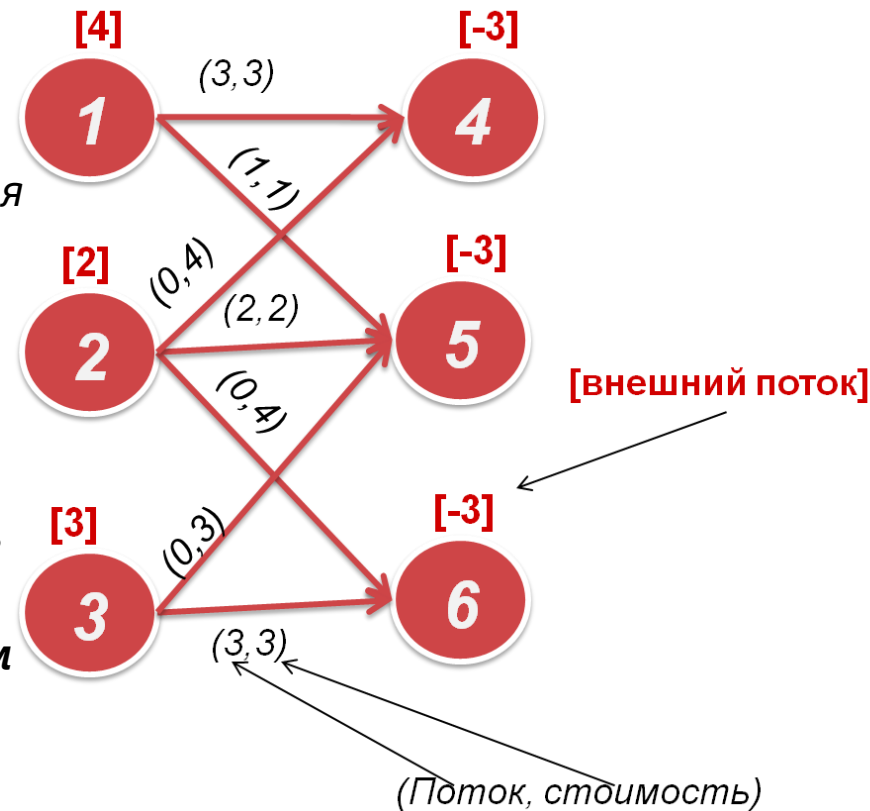
Взаимосвязь между задачами потокowego программирования

- Задача оптимизации некоторого функционала при наличии ограничений называется задачей математического программирования.



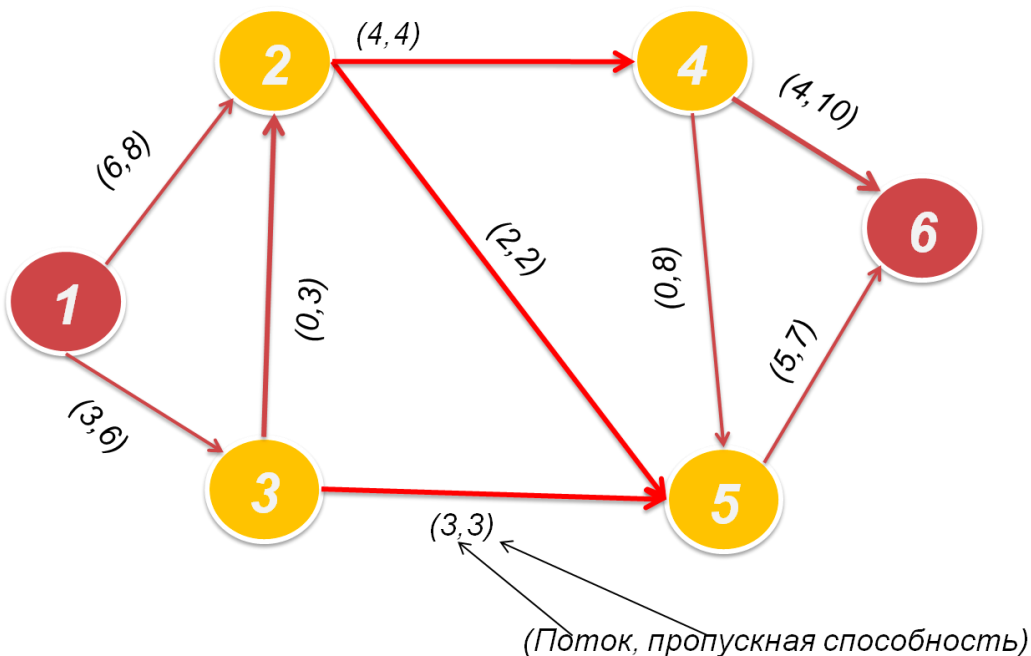
Транспортная задача

- В транспортной задаче используется сеть специального вида: все узлы можно разделить на два таких множества N_1 и N_2 , что все дуги выходящие N_1 из узлов множества, входят в N_2 .
- Задача характеризуется еще тремя отличительными особенностями:
- 1. Каждая дуга сети имеет бесконечную пропускную способность, что позволяет исключить соответствующий параметр из списка дуг.
- 2. Во всех узлах заданы ненулевые фиксированные внешние потоки.
- 3. Сумма внешних потоков по всем узлам равна нулю.



Задача о максимальном потоке

- Единственным параметром является *пропускная способность дуги*. Задача состоит в том, чтобы найти **максимальный поток**, протекающий по дугам сети из заданного узла-источника в заданный узел-сток. Эта задача эквивалентна **задаче о минимальном разрезе**. Они решаются совместно.
- В задаче о минимальном разрезе требуется построить разрез с минимальной суммарной пропускной способностью дуг.

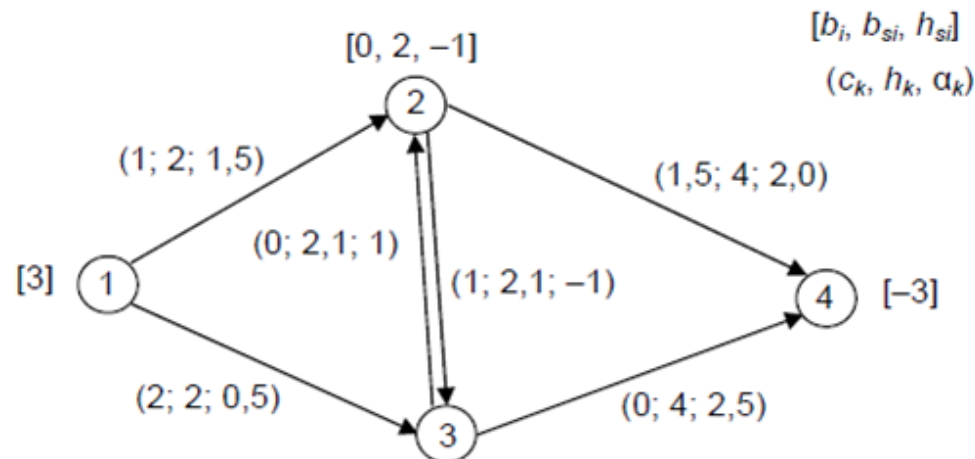


Сети с выигрышами

- Одно из возможных обобщений стандартной задачи о потоке минимальной стоимости представляет собой задача о **сети с выигрышами**.
- Сети с выигрышами, или обобщенные потоковые задачи, образуют важный класс задач потокового программирования, для их решения разработаны эффективные алгоритмы. В указанных задачах *нет условия, согласно которому поток сохраняет свое значение при прохождении по дуге*, напротив, он обычно умножается на некоторый параметр, называемый **коэффициентом выигрыша** (α_k).
- При наличии в сети дуг с параметрами выигрыша, отличными от 1, потоки *изменяют свое значение при прохождении по дугам*. В результате они необязательно будут целочисленным. Поток, входящий в сеть, не обязательно будет равен потоку, выходящему из сети, поэтому условие сохранения потока в источнике можно не формулировать. Однако *условия сохранения потока должны выполняться во всех узлах сети*.
- На дугах сети заданы **пропускная способность, стоимость передачи единицы потока и параметр выигрыша**.

Пример

- Рассмотрим для примера дугу $(1, 3)$, по ней передаются две из трех единиц фиксированного внешнего потока, поступающего в узел 1. Поскольку для дуги $(1, 3)$ параметр выигрыша равен $0,5$, то только одна единица потока ($0,5 \cdot 2 = 1$) поступает в узел 3.
- Аналогично $1,5$ ед. потока выходят из узла 2 по дуге $(2, 4)$, но в узел 4 поступают 3 ед., т. к. параметр выигрыша дуги $(2, 4)$ равен 2. Очевидно, что стандартная потоковая модель является специальным случаем сети с выигрышами, в которой все параметры выигрышей равны 1.



Алгоритм Диница

- Опубликован в 1970 году, имел огромное значение.
- На протяжении 10 лет все достижения в решении задачи о максимальном потоке базировались на методе Диница.
- **Основная идея метода – алгоритм состоит из фаз, на которых поток увеличивается сразу вдоль всех кратчайших цепей определенной длины.**
- Для этого на i -й фазе строится вспомогательная бесконтурная сеть (*layered network*). Эта сеть содержит все увеличивающие цепи, длина которых не превышает k_i , где k_i – длина кратчайшего пути из s в t . Величину k_i называют длиной вспомогательной сети.

Работа алгоритма на i -й итерации:

Шаг 1. Построение вспомогательной сети.

- С помощью поиска в ширину мы движемся из источника сети в сток по допустимым дугам, добавляя их в S_k и увеличивая k . Дуга u добавляется с $c_k(u) = c(u) - f(u)$. Как только достигнут сток сети, он помечается величиной k и k становится «фиксированной». Продолжаем **поиск в ширину**, но он уже не ведется из вершин v , для которых $q(s, v) \geq k$. Таким образом, вспомогательная сеть будет содержать подсеть исходной. Если поиск в ширину не достиг стока, то работа алгоритма прекращается. Если k больше (k может только увеличиваться) k_i , то мы находимся на $i + 1$ фазе с $k_{i+1} = k$.

Шаг 2. Поиск псевдомаксимального потока.

- В построенной нами бесконтурной сети длины k ищется **псевдомаксимальный поток**. Под псевдомаксимальным потоком понимается такой поток f , для которого не существует увеличивающих цепей длины k . Найденный поток переносится в исходную сеть. Затем мы вновь переходим к первому шагу.
- Для построения псевдомаксимального потока используется **поиск в ширину** (с ограничением на длину пути). Пусть на j -ой итерации был найден путь из s в t . Пустим по этому пути поток f_j . Это значит, что как минимум одна дуга вспомогательной сети является насыщенной. Удалим все насыщенные дуги. В результате могут образоваться «тупики»: вершины, из которых не выходит ни одна дуга (кроме стока), вершины, в которые не входит ни одна дуга (кроме источника) и изолированные вершины. Их также следует удалить со всеми инцидентными им дугами. Это в свою очередь может привести к образованию новых тупиков. Корректировка производится, пока во вспомогательной сети не останется ни одного тупика. Изменим пропускные способности оставшихся дуг по формуле $c_k(u) = c_k(u) - f_j(u)$.

Описание алгоритма

- Поиск f_j следует продолжать до тех пор, пока вспомогательная сеть $\neq \emptyset$. Очевидно, что псевдомаксимальный поток $f = \sum f_j$.
- Псевдомаксимальный поток можно хранить в какой-либо структуре, но на практике найденные потоки f_j обычно сразу переносятся в исходную сеть S .
- После нахождения f_i и корректировки сети, поиск в ширину можно продолжать с ближайшей к s не подвергшейся изменениям дуги найденного пути.
- После завершения работы алгоритма исходная сеть будет содержать максимальный поток.

Алгоритм Кинга

- *Работа Кинга, Рао и Таряна (King, Rao и Tarjan) опубликована в 1992 г.*
- Кинг выдвинул более успешную адаптивную стратегию для алгоритма-игрока, улучшив оценку до $O(nm + n^{2+e})$, что давало $O(mn)$ для всех $m > n^{1+e}$. Это позволило увеличить диапазон m / n , для которого на тот момент можно было достичь быстродействия $O(nm)$.
- Кинг немного модифицировал правила игры.

Рассмотрим неориентированный двудольный граф $G = (U, V, E)$. $|U| = |V| = n$, $|E| = m$.

Во время инициализации, алгоритм-игрок назначает дугу каждой вершине из U . Затем игра продолжается по раундам, пока все вершины из V не будут удалены.

Соперник может сделать один из следующих ходов:

- 1. Удалить любую дугу из графа (*edge-kill*), но не получит за это очков.
- 2. Удалить любую вершину из V вместе с инцидентными ей дугами, получив по одному очку за каждую дугу.

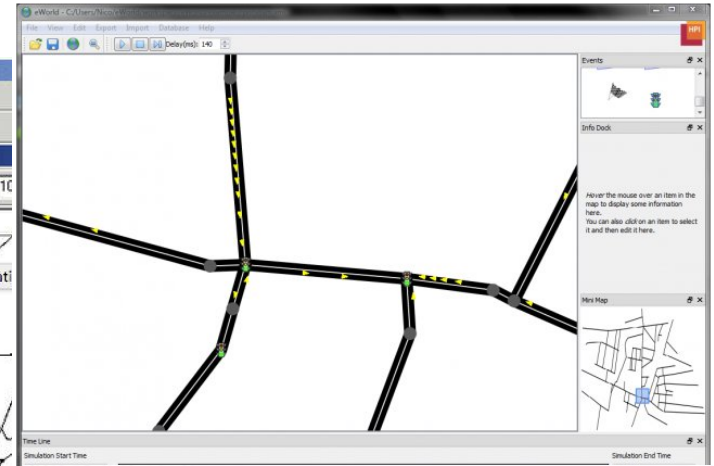
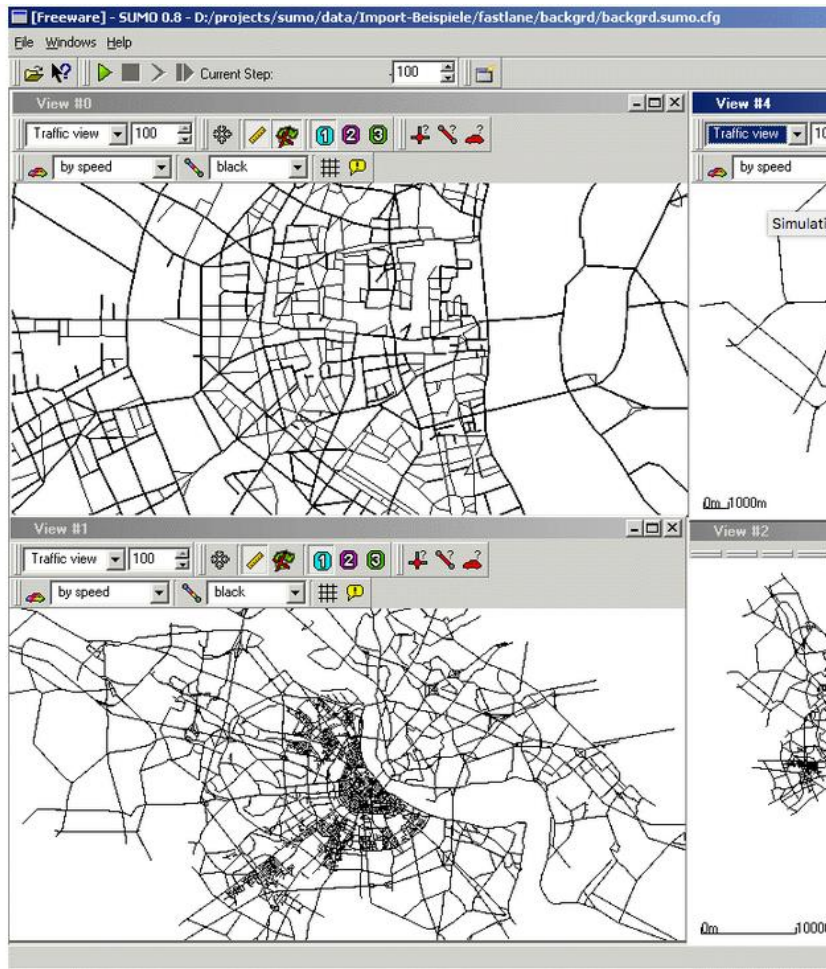
Алгоритм-игрок может сделать любую последовательность следующих ходов:

- 1. Назначить дугу каждой вершине u из U , для которой еще не назначена дуга.
- 2. Назначить вершине новую инцидентную дугу. Старая дуга приобретает обычный статус, а соперник получает дополнительное очко.

Стратегия алгоритма-игрока:

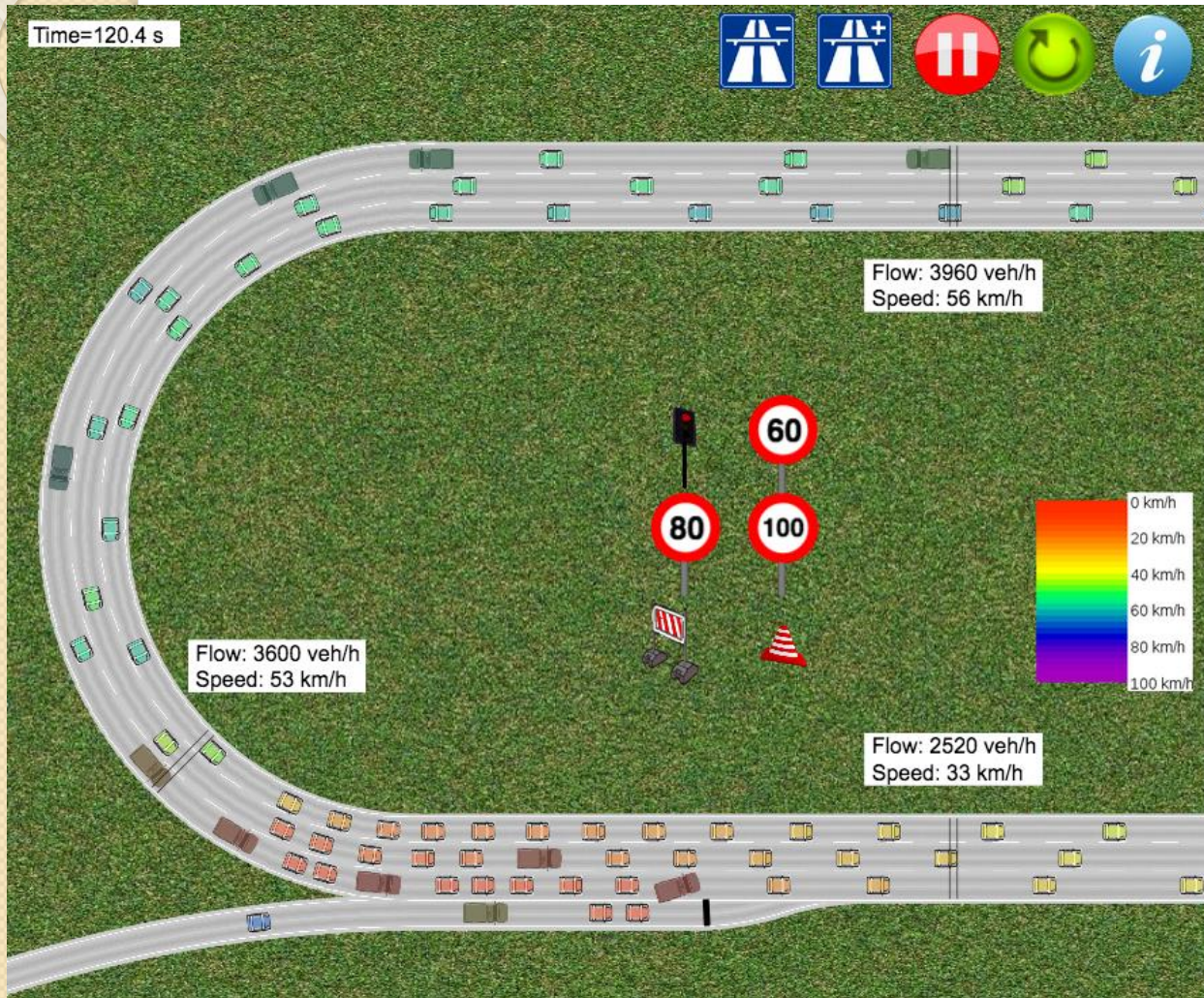
- Пусть l – минимальный коэффициент (k) вершин u . Выделим множество $U' \subseteq U$ вершин, с коэффициентом больше k . Всех v из V сопоставим оценку $r(v)$, равную числу назначенных дуг (u, v) , инцидентных v с минимальным коэффициентом.
- Диапазон значений, которые могут принимать оценки, можно разделить на уровни. Уровень i содержит вершины, оценки которых лежат в интервале $[r_i, r_{i+1})$, где $r_i = 2^i r_0$. Поэтому, вершине из U' достаточно «знать» уровни смежных вершин.
- Когда для вершины $u \in U'$ назначается дуга, алгоритм выбирает дугу инцидентную вершине с минимальным уровнем.

SUMO (simulation of urban mobility)



<http://sumo.sourceforge.net/>

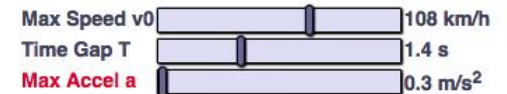
Traffic simulation



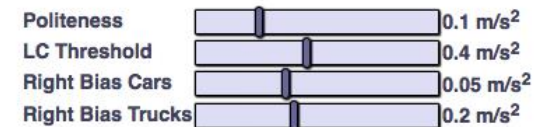
Traffic Flow and General



Car-Following Behavior



Lane-Changing Behavior



- Change the road geometry by dragging
- Click onto the road to disturb traffic flow
- Drag obstacles or construction vehicles to create new bottlenecks
- Drag traffic lights to the road and click on them to toggle between red and light
- Use the info button repeatedly for more info

<http://www.traffic-simulation.de/>



СПАСИБО
ЗА ВНИМАНИЕ